McGill University
School of Computer Science

Odd-One-Out Image Groups

Assignment 3 Report

Prof. Siamak Ravenbakhsh
COMP 551 - Applied Machine Learning

Steven Chang	261239062	
Mujtaba Mirhasan	261213939	23 March 2026
Matthew Lacasse	261183472	

***Abstract*—The task of this project is to identify the outlier image from a set of five grayscale 32×32 images. We propose a deep learning approach combining convolutional feature extraction with a relational scoring mechanism. Each image is processed through a four-layer convolutional encoder using separable convolutions and channel attention, followed by a projector that produces a compact embedding. A scorer then evaluates each image relative to its group using statistical and cosine similarity features, enabling effective odd-one-out detection. To prevent overfitting on the small dataset (3000 training sets) while staying under 25,000 parameters, we applied data standardization, on-the-fly augmentation, dropout, and careful architectural design to respect the parameter count limit. Our final model achieved a public Kaggle score of 79.8%, with 24,071 parameters, significantly improving over the initial score 0.686 (68.6%). Results demonstrate that combining learned feature representations with structured relational features enables accurate outlier detection and further gains could come better generalization and model architecture.**

I. Introduction

The objective of this project is to develop a deep-learning model for identifying an odd-one-out image from a set of five. Unlike standard image classification, where the image is given a label, the model must extract and compare visual features across the entire set to identify the outlier.

The training dataset contains 3000 sets of 5 black and white images of size 32 by 32 pixels, including shapes of all sorts, for which our model is allowed to be trained. The pixel hold only 1 value between 0 and 255 (luminance). Thereafter, our model will be tested on a test set of 2,000 examples to evaluate our performance on examples that our model was never trained on. As a constraint, our model must not exceed 25,000 parameters.

Albeit a simple problem, traditional logistic regression models fail to produce any result better than a random guess (20% accuracy). This is because Logistic Regression lacks locality, and the features that need to be learned cannot be easily expressed as engineered features. Deep learning, specifically convolutional neural networks (CNN) use filters that slide across the image, and learn parameters while maintaining locality. This locality property is useful in many scenarios, like generative AI or object detection and recognition.

By using small sliding windows (called kernels), a convolutional neural network can capture local patterns like curves and color changes. We believe those patterns can be crucial for this scenario of odd-one-out, and is the approach we have taken.

II. Data Processing

Neural networks perform much better when inputs are small and centered around zero. For that reason, we have subtracted the mean and divided by the standard deviation for every picture in the dataset. This method is called standardization, and is generally considered a good method for deep learning.

Continuing, with only 3000 training groups and a maximum of 25,000 parameters, our model has a high risk of overfitting. To combat this, we have done some data augmentation to simulate new entries to train on. The data augmentation is computed on-the-fly at every batch access, i.e. each sample is transformed once per epoch. After taking a look at the PyTorch transformation library, we have decided to use the following augmentation transforms:

TABLE II.1: Data Augmentation and Preprocessing Transforms

Transform	**Parameters**	**Applied To**
ToImage	–	Train & Test
RandomHorizontalFlip	$p = 0.5$	Train
RandomVerticalFlip	$p = 0.5$	Train
RandomRotation	$\pm 15^\circ$	Train
ToDtype	f32, scale=True	Train & Test
Normalize	μ=x_{mean}, σ=x_{std}	Train & Test

For validation, We used 10% of the dataset to monitor overfitting and guide hyperparameter tuning. The splits are random (so different samples are part of the validation set), but are reproducible by fixing the seed, ensuring the same validation set is used across runs. In our setup, the validation set is evaluated every 10 epochs to reduce computational overhead.

III. Methods

A. Architecture

Our model architecture consists of three main components: an encoder, a projector, and a scorer. The encoder is a convolutional feature extractor that processes each image independently using four convolutional blocks. The first layer is a standard 3×3 convolution, followed by batch normalization

to stabilize training and improve convergence. This is then passed through a SiLU activation function. The SiLU activation function was used over ReLU for it's better non-linearity and it's non-flat and non-zero gradient in the negatives.

The batch normalization computes the mean and variance of the activations across each mini-batch. It then normalizes the activations using these statistics and applies two learnable parameters — a scale γ and a shift β — before producing the final output of the layer. This normalization can make

The second, third, and fourth layers use separable convolutions of size 3×3. Each separable convolution consists of a depthwise convolution followed by a pointwise convolution (see Figure A.1 in Appendix A for a visual representation). The depthwise convolution processes each input channel independently with its own 3×3 kernel, preserving the number of channels. The subsequent pointwise convolution then combines these channels using a 1×1 kernel to produce the desired number of output channels. This design significantly reduces the number of parameters compared to standard convolutions while maintaining strong representational power. Layer 3 and 4 both contain a MaxPool2d layer that only keeps the strongest feature maps to reduce the amount of computations needed for speed.

For the depthwise convolution, the formula is:

$$y_{c,i,j} = \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} W_{c,u,v}\, x_{c,i+u,j+v}$$

where

- $y_{c,i,j}$: output at spacial position (i, j) for channel c
- $x_{c,i+u,j+v}$: input at channel c, shifted by kernel indices u, v
- $W_{c,u,v}$: kernel weights for channel c (depthwise: 1 kernel per input channel)
- $K_h = K_w = K$: kernel size (e.g. 3 for 3x3)
- $c \in \{1, 2, ..., C_{in}\}$: input channel index

For the pointwise convolution, the formula is:

$$y_{k,i,j} = \sum_{c=1}^{C_{\text{in}}} W_{k,c}\, x_{c,i,j}$$

where

- $y_{k,i,j}$: output at spacial position (i, j) or output channel k
- $x_{c,i,j}$: input at spacial location (i, j), channel c
- $W_{k,c}$: weight connecting input channel c to output channel k
- C_{in}: number of input channel index
- $k \in \{1, 2, ..., C_{out}\}$: output channel index

In the third and fourth layers, we additionally introduce a channel attention mechanism. This module starts by computing the mean of its feature map, for each channel c. the means is given by $\bar{x} = \frac{1}{HW} \sum_{h=1}^{H} \sum_{w=1}^{W} x_{c,h,w}$. This produces a vector of size equal to the number of channels (72 in our case). This vector is then compressed to a lower-dimensional representation of size 6 using a linear layer, followed by a SiLU activation. It is then expanded back to 72 dimensions using another linear layer. This layer helps with overfitting

After the encoder, the spatial dimensions are removed using a combination of global average pooling and global max pooling, whose outputs are averaged to form a compact feature representation for each image. These features are then passed through a projector, which consists of two fully connected layers. The first layer reduces the feature dimension from 72 to 64 and is followed by batch normalization and a SiLU activation. The second layer further projects the features into a lower-dimensional embedding space of size PROJ_DIM $= 22$.

Finally, the scorer operates on groups of images. For each image embedding, we compute several statistical features relative to the group, including a normalized z-score, an absolute z-score, and a cosine similarity with the group direction.

To calculate the z-score for an image $i \in \{1..5\}$ in a group of 5 images with embedding vector $\mathbf{f}_i \in \mathbb{R}^{22}$, we first calculate the leave-one-out mean, given by the following formula:

$$\bar{\mathbf{f}}_{-i} = \frac{1}{5-1} \sum_{\substack{j=1 \\ j \neq i}}^{5} \mathbf{f}_j = \frac{\mathbf{f}_1 + \mathbf{f}_2 + \mathbf{f}_3 + \mathbf{f}_4 + \mathbf{f}_5 - \mathbf{f}_i}{4}$$

After this, we calculate the group standard deviation:

$$\sigma = \sqrt{\frac{1}{5} \sum_{j=1}^{5} (\mathbf{f}_j - \bar{\mathbf{f}})^2 + \epsilon}, \quad \text{where } \bar{\mathbf{f}} = \frac{1}{5} \sum_{j=1}^{5} \mathbf{f}_j$$

The z-score and the absolute z-score can then be obtained with:

$$\mathbf{z}_i = \frac{\mathbf{f}_i - \bar{\mathbf{f}}_{-i}}{\sigma} \in \mathbb{R}^{22}$$

$$|\mathbf{z}_i| \in \mathbb{R}^{22}$$

For the cosine similarity, we have to normalize each embedding:

$$\hat{\mathbf{f}}_i = \frac{\mathbf{f}_i}{\|\mathbf{f}_i\|_2}$$

We then need to compute the leave-one-out direction. The numerator is the sum of all the normalized embeddings except i. The denominator divides by its L2 norm, turning the fraction into a unit vector. This unit vector represents the "direction" of the group without the i-th image. The formula is:

$$\hat{\mathbf{d}}_{-i} = \frac{\sum_{j \neq i} \hat{\mathbf{f}}_j}{\left\| \sum_{j \neq i} \hat{\mathbf{f}}_j \right\|_2}$$

We do a simple dot product to get the cosine similarity in the end:

$$\text{cos_sim}_i = \hat{\mathbf{f}}_i \cdot \hat{\mathbf{d}}_{-i} \in \mathbb{R}$$

The original embedding, the z-score, the absolute z-score, and the cosine similarity are concatenated, resulting in a vector of size PROJ_DIM $\times 3 + 1 \Rightarrow 22 \times 3 + 1 = 67$.

This combined representation is passed through a small multilayer perceptron (MLP), which produces a single logit score for each image. The input to the MLP is a 67-dimensional feature vector that encapsulates all information for that image. The first fully connected layer reduces this vector to 24 dimensions, followed by a SiLU activation and dropout for regularization. A second linear layer then maps the 24-dimensional vector to a single scalar logit. The model is trained to assign higher scores to images that are outliers within their group.

B. Total Number of Parameters

The first layer is a standard convolution with a 3x3 kernel, no bias. The number of learnable parameters is obtained by $C_{out} \times C_{in} \times K^2 = 32 \times 1 \times 3 \times 3 = 288$. The batch normalization then adds 2 learnable parameters per channel, 32×2, giving a total of $288 + 64 = 352$ parameters for the first layer.

The second layer is a separable convolution (SepConv) from 32 to 64 channels. The depthwise convolution has one kernel per input channel: $32 \times 3 \times 3 = 288$ parameters. The pointwise 1×1 convolution mixes channels to produce $32 \times 64 = 2048$ parameters. Batch normalization adds $64 \times 2 = 128$ parameters. The total for the second layer is therefore $288 + 2048 + 128 = 2464$.

The third layer is a SepConv with a channel attention layer (Comp). The depthwise convolution has $64 \times 3^2 = 576$ learnable parameters and the pointwise has $64 \times 72 = 4608$ learnable parameters. The Comp layer begins by compressing from 72 channels to 6 channels and this gives $72 \times 6 = 432$ parameters. Going from 6 to 72 channels also gives 432 parameters ($6 \times 72 = 432$). The total is $576 + 4608 + 144 + 432 + 432 = 6192$ learnable parameters in this layer.

The fourth layer is another SepConv with Comp. The depthwise convolution has $72 \times 3^2 = 648$ and pointwise has $72 \times 72 = 5184$. Batch normalization adds $72 \times 2 = 144$. The Comp layer add 432 for compression and another 432 for decompression (reducing 72 channels to 6 and decompressing from 6 to 72). The total number of parameters for the fourth layer is $648 + 5184 + 144 + 432 + 432 = 6840$ learnable parameters.

Then, the projector is made of 2 linear layers. The first layer has $72 \times 64 + 64 = 4672$ parameters (weights and bias). 1D batch normalization adds $64 \times 2 = 128$. The second linear layer then goes from 64 feature to 22 features, so there are $64 \times 22 + 22 = 1410$ parameters. The total adds up to $4672 + 128 + 1410 = 6210$

Finally, for the scorer, there are 2 linear layers. The input is a vector of size PROJ_DIM $\times 3 + 1$, so $22 \times 3 + 1 = 67$. The first linear layer has $67 \times 24 + 24 = 1632$. The last layer goes from 24 inputs to 1 output, so $24 \times 1 + 1 = 25$ parameters. The total of this layer is 1657.

The sum of all layers gives $352 + 2464 + 6192 + 6840 + 6210 + 1657 = 23715$. PyTorch returns 24,071 parameters, which could be from the internal biases the layers use, however in both cases, (theoretical, and experimentally), there are fewer than 25k parameters.

C. Loss function and Optimization

The model is trained using a cross-entropy loss with label smoothing of 0.1, which helps prevent overconfident predictions. Optimization is carried out with the

AdamW optimizer, which combines adaptive learning rates with weight decay to improve generalization. To adjust the learning rate dynamically, we employ a cosine annealing scheduler with warm restarts. The first restart occurs after 100 epochs ($T_0 = 100$), and each subsequent cycle is four times longer than the previous one ($T_mult = 7$), allowing the optimizer to periodically explore new regions of the loss landscape and avoid getting stuck in local minima. The learning rate is set so that it cannot go lower than 1×10^{-6}.

D. Hyperparameters

Key hyperparameters for training are as follows: the batch size is set to 64, and training continues until manually interrupted. The batch size of 64 helps the model get out of local minima easily, and we observed a faster convergence. Dropout of 0.425 is applied in the scorer to prevent overfitting. The convolutional layers have dimensions $L_1 = 32$, $L_2 = 64$, $L_3 = 72$, and $L_4 = 72$, while the projector reduces features from 72 to 64 and then to a final embedding size of 22. The scorer uses an intermediate layer of size 24.

Then, for the training loop, the learning rate starts at 0.004 with a weight decay of 3×10^{-5} and follows a cosine annealing schedule with warm restarts to stabilize training.

IV. Results & Analysis

The report is based on the version of various models that scored the highest on the Kaggle leaderboard (79.8%).

In the beginning, our first OutlierModel was made of an encoder of only 3 standard convolution layers (with batch normalization, LeakyReLU and MaxPool after activation). The output of the CNN encoder is flattened into a 1D feature vector per sample. It is then used as input to a projector and a scorer. The layer widths had different values than the current architecture. The engineered featured were also different. With that model, we ran the training loop for 240 epochs and got a score of 0.68600. We then noticed that the model was underfitting because the training loss was still going down:

So, from that we ran the model for longer and got to 0.72000. The training loss later stabilized at around epochs 280 to 300.

After architectural changes like replacing LeakyReLU with SiLU and using SepConv layers to save parameters, we ended up with a better model.

Epoch	Training Loss
120	13.40
140	12.30
...	...
220	11.38
240	11.15

TABLE IV.1: Training loss at selected epochs. The steadily decreasing loss indicates that the model was underfitting.

Epoch	Training Loss
280	8.75
290	8.77
...	...
330	8.50
340	8.77

TABLE IV.2: Training loss at later epochs, showing that the decrease has stalled and the model is converging.

For our most recent model, the loss appeared to have plateaued around epoch 530. Table IV.3 summarizes the training and validation performance along with the public Kaggle score at selected later epochs, demonstrating that the model had effectively converged. For our final model, both layer 1 and 2 had their MaxPool2d layer removed. After further testing, we noticed that, considering the image is already pretty small, our model achieved better results by removing the MaxPool2d and training more on the 32×32 feature maps.

Epoch	Train Loss	Val Loss	Val Acc	Public Score
360	0.181	0.101	0.967	0.754
370	0.168	0.117	0.953	0.747
380	0.176	0.147	0.960	0.750
...	...	...	...	...
430	0.176	0.133	0.960	0.748
440	0.187	0.098	0.967	0.752
450	0.177	0.115	0.973	0.753

TABLE IV.3: Final training and validation performance of the best OutlierModel.

From Table IV.3, we observe that the training loss managed to stabilize after epoch 530. The validation loss is pretty low and the validation accuracy is really high (as high as 97%). The final public Kaggle score reached 79.8%, significantly improving over the initial model (0.686).

These results demonstrate that the architectural improvements, combined with improved engineered hardcoded features and better projector-scorer pipeline, increased the model's capacity to detect

outlier images while controlling overfitting. In particular, the use of SepConv significantly reduces the number of parameters compared to standard convolutions. This reduction allows the model to allocate capacity more efficiently, enabling the use of deeper layers and richer feature representations without increasing computational cost. As a result, the model can learn more fine-grained patterns in the data, which contributes to improved outlier detection performance and a higher final Kaggle score. The use of SiLU activations can help convergence and the feature normalization helps optimization and improves generalization. Furthermore, the removal of initial MaxPool2d layers made the computations a lot more lengthy, however, this change managed to boost our score from 77.3% all the way up to 79.8%

In summary, ablation studies comparing the early baseline and final models show that each change — activation function, convolution type, and feature engineering — had a measurable positive effect on performance, leading to a better version of the OutlierModel.

V. Discussion & Conclusion

The final performance of the model can be attributed to a combination of architectural improvements and feature engineering. In particular, the usage of SepConv layers significantly reduced the number of parameters compared to standard convolutions. This allowed the model to use its capacity more efficiently, enabling deeper and more expressive representations without increasing computational cost. As a result, the model was able to capture more fine-grained visual patterns, which contributed to improved outlier detection performance. Additionally, the change from LeakyReLU to SiLU activations improved training stability and convergence, leading to higher final accuracy.

Another important factor was the inclusion of engineered hardcoded features. These features explicitly capture how different each image is relative to the rest of the group, which aligns closely with the objective of outlier detection. Combined with the projector-scorer pipeline, this allowed the model to effectively separate representation learning from decision making, further improving performance.

Despite these improvements, several limitations remain. The relatively small dataset makes the model prone to overfitting and limits its ability to generalize. Although validation accuracy is high (going up to 97%+), there is still a huge gap between the Kaggle test score and the validation accuracy. This could be caused by the fact that the public test set may contain slightly different images, styles, or noise than the training set. Also, since the training data and the test data includes mislabeled examples, the model can be confused, because it learned patterns that don't generalize perfectly to those cases. Furthermore, relying on manually engineered features may reduce the model's adaptability to other datasets, especially if the test set comes from another distribution. The training process also showed signs of plateauing after around 530 epochs, indicating that further training yields diminishing returns.

In terms of future work, several directions could be explored. Performing changes to the width of the layers, the activation functions, and the engineered features may result in a higher score. Increasing the dataset size or applying stronger data augmentation techniques would likely improve generalization. One thing that can be tried is to play with the contrast of the grayscale images. Additional hyperparameter tuning, including the dimensions of the projector and scorer or the dropout rate, could further refine performance. One advanced way to further upgrade upgrade our model is to use Karpathy's Autoresearch, newly released in March 2026. That LLM would try out many architectures and hyperparameters to improve our model. However, this may be too far fetched for this assignment.

Overall, the results suggest that combining learned feature representations with structured relational features is an effective strategy for outlier detection, though further improvements could be achieved through better generalization and more careful examination of how each part of the model contributes to its performance and accuracy.

VI. Statement of Contributions

Matthew did most of the coding part. Mujtaba and Steven did most of the report.

APPENDIX

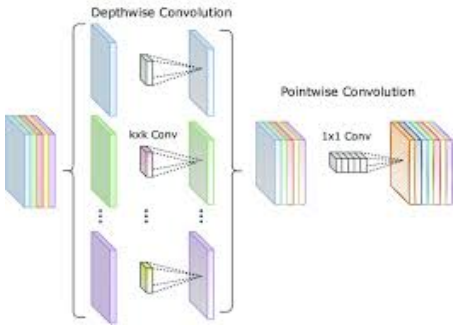

Fig. A.1: Depthwise + Pointwise convolution schematic.